A Practical Guide to Actor critic algorithm

TOPIC -- ACTOR CRITIC ALGORITHM

-- 01 --

γ j ∑ n = 1 ∞ λ n – 1 1 – λ · (∑ k = 0 n – 1 γ k R j + k + γ n V π θ (S j + n) – V π θ (S j)) {\textstyle \gamma ^{j}\sum _{n=1}^{\infty }{\frac {\lambda ^{n-1}}{1-\lambda }}\cdot \left(\sum _{k=0}^{n-1}\gamma ^{k}R_{j+k}+\gamma ^{n}V^{\pi _{\theta }}(S_{j+n})-V^{\pi _{\theta }}(S_{j})\right)} : TD(λ) learning, also known as GAE (generalized advantage estimate). This is obtained by an exponentially decaying sum of the TD(n) learning terms.

-- 02 --

γ j (R j + γ R j + 1 + γ 2 V π θ (S j + 2) – V π θ (S j)) {\textstyle \gamma ^{j}\left(R_{j}+\gamma R_{j+1}+\gamma ^{2}V^{\pi _{\theta }}(S_{j+2})-V^{\pi _{\theta }}(S_{j})\right)} : TD(2) learning.

-- 03 --

γ j (R j + γ V π θ (S j + 1) – V π θ (S j)) {\textstyle \gamma ^{j}\left(R_{j}+\gamma V^{\pi _{\theta }}(S_{j+1})-V^{\pi _{\theta }}(S_{j})\right)} : TD(1) learning.

-- 04 --

Konda, Vijay R.; Tsitsiklis, John N. (January 2003). "On Actor-Critic Algorithms". SIAM Journal on Control and Optimization. 42 (4): 1143–1166. doi:10.1137/S0363012901385691. ISSN 0363-0129. Sutton, Richard S.; Barto, Andrew G. (2018). Reinforcement learning: an introduction. Adaptive computation and machine learning series (2 ed.). Cambridge, Massachusetts: The MIT Press. ISBN 978-0-262-03924-6. Bertsekas, Dimitri P. (2019). Reinforcement learning and optimal control (2 ed.). Belmont, Massachusetts: Athena Scientific. ISBN 978-1-886529-39-7. Grossi, Csaba (2010). Algorithms for Reinforcement Learning. Synthesis Lectures on Artificial Intelligence and Machine Learning (1 ed.). Cham: Springer International Publishing. ISBN 978-3-031-00423-0. Grondman, Ivo; Busoniu, Lucian; Lopes, Gabriel A. D.; Babuska, Robert (November 2012). "A Survey of Actor-Critic Reinforcement Learning: Standard and Natural Policy Gradients". IEEE Transactions on Systems, Man, and Cybernetics - Part C: Applications and Reviews. 42 (6): 1291–1307. Bibcode:2012ITHMS..42.1291G. doi:10.1109/TSMCC.2012.2218595. ISSN 1094-6977.

-- 05 --

Variants Asynchronous Advantage Actor-Critic (A3C): Parallel and asynchronous version of A2C. Soft Actor-Critic (SAC): Incorporates entropy maximization for improved exploration. Deep Deterministic Policy Gradient (DDPG): Specialized for continuous action spaces.

-- 06 --

Actor The actor uses a policy function π (a | s) {\displaystyle \pi (a|s)} , while the critic estimates either the value function V (s) {\displaystyle V(s)} , the action-value Q-function Q (s , a) , {\displaystyle Q(s,a),} the advantage function A (s , a) {\displaystyle A(s,a)} , or any combination thereof. The actor is a parameterized function π θ {\displaystyle \pi _{\theta }} , where θ {\displaystyle \theta } are the parameters of the actor. The actor takes as argument the state of the environment s {\displaystyle s} and produces a probability distribution π θ (· | s) {\displaystyle \pi _{\theta }(\cdot |s)} . If the action space is discrete, then ∑ a π θ (a | s) = 1 {\displaystyle \sum _{a}\pi _{\theta }(a|s)=1} . If the action space is continuous, then ∫ a π θ (a | s) d a = 1 {\displaystyle \int _{a}\pi _{\theta }(a|s)da=1} . The goal of policy optimization is to improve the actor. That is, to find some θ {\displaystyle \theta } that maximizes the expected episodic reward J (θ) {\displaystyle J(\theta)} : J (θ) = E π θ [∑ t = 0 T γ t r t] {\displaystyle J(\theta)=\mathbb {E} _{\pi _{\theta }}\left[\sum _{t=0}^{T}\gamma ^{t}r_{t}\right]} where γ {\displaystyle \gamma } is the discount factor, r t {\displaystyle r_{t}} is the reward at step t {\displaystyle t} , and T {\displaystyle T} is the time-horizon (which can be infinite). The goal of policy gradient method is to optimize J (θ) {\displaystyle J(\theta)} by gradient ascent on the policy gradient ∇ J (θ) {\displaystyle \nabla J(\theta)} . As detailed on the policy gradient method page, there are many unbiased estimators of the policy gradient: ∇ θ J (θ) = E π θ [∑ 0 ≤ j ≤ T ∇ θ ln ■ π θ (A j | S j) · Ψ j | S 0 = s 0] {\displaystyle \nabla _{\theta }J(\theta)=\mathbb {E} _{\pi _{\theta }}\left[\sum _{0\leq j\leq T}\nabla _{\theta }\ln \pi _{\theta }(A_{j}|S_{j})\cdot \Psi _{j}{\Big |}S_{0}=s_{0}\right]} where Ψ j {\textstyle \Psi _{j}} is a linear sum of the following:

-- 07 --

The actor-critic algorithm (AC) is a family of reinforcement learning (RL) algorithms that combine policy-based RL algorithms such as policy gradient methods, and value-based RL algorithms such as value iteration, Q-learning, SARSA, and TD learning. An AC algorithm consists of two main components: an "actor" that determines which actions to take according to a policy function, and a "critic" that evaluates those actions according to a value function. Some AC algorithms are on-policy, some are off-policy. Some apply to either continuous or discrete action spaces. Some work in both cases.

-- 08 --

γ j (∑ k = 0 n − 1 γ k R j + k + γ n V π θ (S j + n) − V π θ (S j)) {\textstyle \gamma ^{j}\left(\sum _{k=0}^{n-1}\gamma ^{k}R_{j+k}+\gamma ^{n}V^{\pi _{\theta }}(S_{j+n})-V^{\pi _{\theta }}(S_{j})\right)} : TD(n) learning.

-- 09 --

γ j ∑ j ≤ i ≤ T (γ i − j R i) − b (S j) {\textstyle \gamma ^{j}\sum _{j\leq i\leq T}(\gamma ^{i-j}R_{i})-b(S_{j})} : the REINFORCE with baseline algorithm. Here b {\displaystyle b} is an arbitrary function.

-- 10 --

Critic In the unbiased estimators given above, certain functions such as V π θ , Q π θ , A π θ {\displaystyle V^{\pi _{\theta }},Q^{\pi _{\theta }},A^{\pi _{\theta }}} appear. These are approximated by the critic. Since these functions all depend on the actor, the critic must learn alongside the actor. The critic is learned by value-based RL algorithms. For example, if the critic is estimating the state-value function V π θ (s) {\displaystyle V^{\pi _{\theta }}(s)} , then it can be learned by any value function approximation method. Let the critic be a function approximator V φ (s) {\displaystyle V_{\phi }(s)} with parameters φ {\displaystyle \phi } . The simplest example is TD(1) learning, which trains the critic to minimize the TD(1) error: δ i = R i + γ V φ (S i + 1) − V φ (S i) {\displaystyle \delta _{i}=R_{i}+\gamma V_{\phi }(S_{i+1})-V_{\phi }(S_{i})} The critic parameters are updated by gradient descent on the squared TD error: φ ← φ − α ∇ φ (δ i) 2 = φ + α δ i ∇ φ V φ (S i) {\displaystyle \phi \leftarrow \phi -\alpha \nabla _{\phi }(\delta _{i})^{2}=\phi +\alpha \delta _{i}\nabla _{\phi }V_{\phi }(S_{i})} where α {\displaystyle \alpha } is the learning rate. Note that the gradient is taken with respect to the φ {\displaystyle \phi } in V φ (S i) {\displaystyle V_{\phi }(S_{i})} only, since the φ {\displaystyle \phi } in γ V φ (S i + 1) {\displaystyle \gamma V_{\phi }(S_{i+1})} constitutes a moving target, and the gradient is not taken with respect to that. This is a common source of error in implementations that use automatic differentiation, and requires "stopping the gradient" at that point. Similarly, if the critic is estimating the action-value function Q π θ {\displaystyle Q^{\pi _{\theta }}} , then it can be learned by Q-learning or SARSA. In SARSA, the critic maintains an estimate of the Q-function, parameterized by φ {\displaystyle \phi } , denoted as Q φ (s , a) {\displaystyle Q_{\phi }(s,a)} . The temporal difference error is then calculated as δ i = R i + γ Q θ (S i + 1 , A i + 1) − Q θ (S i , A i) {\displaystyle \delta _{i}=R_{i}+\gamma Q_{\theta }(S_{i+1},A_{i+1})-Q_{\theta }(S_{i},A_{i})} . The critic is then updated by θ ← θ +

α δ i ∇ θ Q θ (S i , A i) {\displaystyle \theta \leftarrow \theta +\alpha \delta _{i}\nabla _{\theta }Q_{\theta }(S_{i},A_{i})} The advantage critic can be trained by training both a Q-function Q φ (s , a) {\displaystyle Q_{\phi }(s,a)} and a state-value function V φ (s) {\displaystyle V_{\phi }(s)} , then let A φ (s , a) = Q φ (s , a) − V φ (s) {\displaystyle A_{\phi }(s,a)=Q_{\phi }(s,a)-V_{\phi }(s)} . Although, it is more common to train just a state-value function V φ (s) {\displaystyle V_{\phi }(s)} , then estimate the advantage by A φ (S i , A i) ≈ ∑ j ∈ 0 : n − 1 γ j R i + j + γ n V φ (S i + n) − V φ (S i) {\displaystyle A_{\phi }(S_{i},A_{i})\approx \sum _{j\in 0:n-1}\gamma ^{j}R_{i+j}+\gamma ^{n}V_{\phi }(S_{i+n})-V_{\phi }(S_{i})} Here, n {\displaystyle n} is a positive integer. The higher n {\displaystyle n} is, the more lower is the bias in the advantage estimation, but at the price of higher variance. The Generalized Advantage Estimation (GAE) introduces a hyperparameter λ {\displaystyle \lambda } that smoothly interpolates between Monte Carlo returns (λ = 1 {\displaystyle \lambda =1} , high variance, no bias) and 1-step TD learning (λ = 0 {\displaystyle \lambda =0} , low variance, high bias). This hyperparameter can be adjusted to pick the optimal bias-variance trade-off in advantage estimation. It uses an exponentially decaying average of n-step returns with λ {\displaystyle \lambda } being the decay strength.